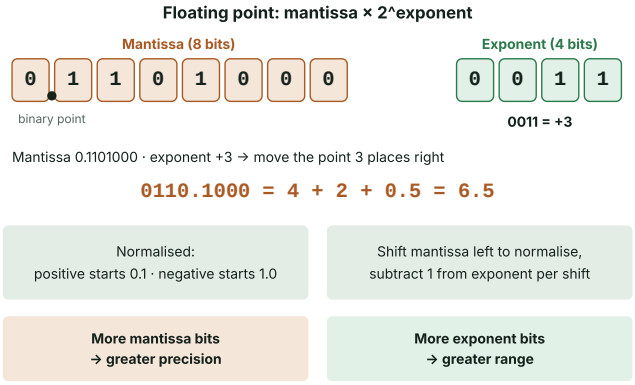
Real numbers: fixed-point and floating-point

FD03
FACT FILE

Specification: students should be able to

- Represent real numbers in fixed-point binary and convert between fixed-point and denary
- Explain floating-point representation using a two's complement mantissa and exponent
- Convert between denary and normalised floating-point binary
- Normalise floating-point numbers and explain the purpose of normalisation
- Explain the trade-off between range and precision and compare fixed-point with floating-point
- Describe rounding errors, overflow and underflow with real numbers

Key words: real number, fixed point, binary point, floating point, mantissa, exponent, normalisation, precision, range, rounding error, absolute error, relative error, overflow, underflow, IEEE 754



A floating-point number with an 8-bit two's complement mantissa and 4-bit exponent; the exponent of 3 moves the binary point three places right to give 6.5

Why real numbers need special treatment

Many quantities that programs handle are not whole numbers: distances, temperatures, prices, times and scientific measurements. A computer stores everything in a fixed number of bits, so it needs an agreed way to represent the fractional part. Binary place value can be extended to the right of a binary point, where the columns are worth a half, a quarter, an eighth, a sixteenth and so on. There are two ways to decide where that point goes. In fixed-point representation the position of the point is agreed in advance and never changes. In floating-point representation the number is stored as a mantissa and an exponent, and the exponent says how far to move the point, so it can 'float' to suit the size of the number. Both methods trade off range, which is how large and how small the stored values can be, against precision, which is how finely values can be distinguished. Neither can represent every real number exactly, because there are infinitely many real numbers and only a finite number of bit patterns. Understanding these limits explains why programs sometimes give answers such as 0.9999999999999999 instead of 1.

Fixed-point representation

In fixed-point binary a set number of bits represent the whole part and the rest represent the fraction. With four bits either side of the point, 0110.1010 means 4 + 2 + 0.5 + 0.125 = 6.625. To convert denary to fixed point, split the value into powers of two: 9.375 is 8 + 1 + 0.25 + 0.125, which is 1001.0110. Negative values can be stored by using two's complement, so that in this format the leftmost bit is worth −8. The precision is the value of the smallest column, here 1/16 or 0.0625, and it is the same across the whole range, which is described as constant absolute precision. The range is limited: the largest unsigned value with four whole-number bits is 15.9375. Moving the point to the right increases the range but leaves fewer bits for the fraction, so precision falls; moving it left does the reverse. Fixed point is simple and fast because the processor can use ordinary integer circuits, which makes it suitable for systems with a known, narrow range, such as audio samples, some sensors and embedded controllers.

Floating-point representation

Floating point is the binary version of standard form. A number is stored as a mantissa multiplied by two raised to the power of an exponent. In the format used in this course both parts are in two's complement. The mantissa is a fraction with the binary point immediately after its first bit, which is the sign bit and is worth −1; the exponent is a two's complement integer. To convert to denary, first work out the exponent, then move the binary point that many places – right for a positive exponent, left for a negative one – and finally add up the place values, remembering that the leftmost bit of the result has a negative weight if the mantissa is negative. For example, 0.1101000 with exponent 0011 becomes 0110.1000, which is 6.5. Floating point allows a very wide range of values, from tiny fractions to huge numbers, to be stored in the same number of bits. The cost is that the arithmetic is more complex and slower than for fixed point, and the precision is relative: large numbers are stored with large gaps between neighbouring values.

Normalisation

The same value can be written in floating point in more than one way. For example, 0.0010110×2^{6} and 0.1011000×2^{4} both equal 11. The first version wastes two bits of the mantissa on leading zeros that add no information. A floating-point number is normalised when the first two bits of the mantissa differ: 0.1 for positive numbers and 1.0 for negative numbers. To normalise, shift the mantissa left until this is true, filling with zeros on the right, and subtract one from the exponent for every place shifted, so that the value is unchanged. For a negative number, the redundant bits are repeated leading 1s: 1.1101000×2^{3} becomes 1.0100000×2^{1} after two shifts. Normalisation has two benefits. It makes the most of the available mantissa bits, so the precision is as high as possible, and it ensures each value has exactly one representation, which makes comparing and processing numbers simpler. When converting denary to floating point, aim for a normalised result directly by moving the point until the number starts 0.1, then counting the places moved to get the exponent.

Range and precision

With a fixed total number of bits, every bit given to the mantissa is a bit taken from the exponent. More mantissa bits increase precision: more significant bits are kept, so values can be stored more exactly and the gaps between representable numbers are smaller. More exponent bits increase range: the binary point can be moved further, so both very large and very small magnitudes can be stored. With an 8-bit mantissa and a 4-bit exponent the largest positive value is 0.1111111×2^{7}, which is 127, and the smallest positive normalised value is 0.1×2^{-8}, about 0.00195. Re-dividing the same twelve bits as six and six raises the largest value to just under 2^{31}, but leaves only five significant bits after the sign. Designers choose the split to suit the application: scientific calculations need a wide range, while measurements that must be distinguished finely need precision. Real computers use the IEEE 754 standard, where a 32-bit float uses 8 exponent bits and 23 mantissa bits and a 64-bit double uses 11 and 52. It differs in detail from the exam format but follows the same trade-off.

Rounding errors, overflow and underflow

Many simple denary fractions cannot be stored exactly in binary. Just as one third recurs in denary, one tenth recurs in binary as 0.000110011..., so it must be cut off and stored as the nearest available value. The difference between the true and stored value is the absolute error; dividing it by the true value gives the relative error. In a 4.4 fixed-point format, 0.1 would be rounded to the nearest step, 0.125, an absolute error of 0.025 and a relative error of 25 per cent. Errors grow when calculations are repeated: adding 0.1 ten times in a C# double gives 0.9999999999999999, so a test for equality with 1.0 fails. For this reason programmers compare real numbers within a small tolerance, and financial software stores money as whole pence or uses a decimal type. Overflow happens when a result's magnitude is too large for the exponent field to hold. Underflow happens when a result is so close to zero that the exponent cannot be made negative enough, so it is stored as zero. Both can cause wrong results or errors if they are not detected.

Key terms from the mark schemes

Real number
A number that can have a fractional part, such as 6.75 or −0.3125.

Binary point
The point separating the whole-number bits from the fractional bits, whose place values are ½, ¼, ⅛ and so on.

Fixed-point representation
A way of storing real numbers in which the binary point is at an agreed, unchanging position.

Floating-point representation
A way of storing real numbers as a mantissa multiplied by 2 to the power of an exponent, allowing a wide range of values.

Mantissa
The part of a floating-point number that holds its significant digits.
`Credited: significand`

Normalisation
Adjusting a floating-point number so the first two bits of the mantissa differ, maximising precision and giving a unique representation.

Range (floating point)
The span between the largest and smallest magnitudes that can be represented, determined mainly by the exponent bits.

Underflow
An error that occurs when a value is too close to zero to be represented with the available exponent.

Exponent
The part of a floating-point number that states how many places, and in which direction, the binary point is moved.

Precision
How finely values can be distinguished, determined by the number of significant bits (mantissa bits).

Rounding error
The difference between a real value and the nearest value that can be stored in the available bits.

C C# example

`Decoding floating point in C#`

```
string m = "01101000";   // mantissa 0.1101000
string e = "0011";       // exponent
double mant = -(m[0] - '0');          // sign bit is worth -1
for (int i = 1; i < m.Length; i++)
    mant += (m[i] - '0') * Math.Pow(2, -i);
int exp = Convert.ToInt32(e, 2);
if (e[0] == '1') exp -= 1 << e.Length;   // two's complement exponent
Console.WriteLine(mant * Math.Pow(2, exp)); // 6.5
```

Exam focus

Floating-point conversions are marked step by step, so write the exponent in denary first, then show the mantissa with the binary point moved, then the place-value sum and the final answer. Remember that the leftmost bit of a negative mantissa is negative, and that a negative exponent moves the point left. When asked to normalise, give both the new mantissa and the new exponent in binary, padded to the stated number of bits, and check that the value has not changed. When converting a negative denary value, find the positive mantissa first, then take its two's complement; the exponent does not change. Questions about changing the split of bits need the direction of change for both range and precision, for example 'more exponent bits increase the range but reduce precision because there are fewer mantissa bits'. 'Explain why numbers are normalised' needs both maximum precision and a unique representation. Avoid vague claims that one method is 'more accurate' without linking it to the number of significant bits, and do not confuse underflow with a negative number.

? Check yourself

1. State what is meant by the mantissa of a floating-point number. [1]
2. Convert the fixed-point number 0101.1100 (4 bits before and 4 bits after the binary point) to denary. Show your working. [2]
3. Represent 3.625 in unsigned fixed-point binary using 4 bits before and 4 bits after the binary point. [2]

Answers: 1. The part of a floating-point number that holds the significant digits/bits of the value (1) 2. 4 + 1 + 0.5 + 0.25 shown (1) / = 5.75 (1) 3. 3 = 0011 (1) / .625 = .1010, giving 0011.1010 (1)